Speeding up machine learning models with compute kernels

Daniël de Kok



About me

- 2008-2012: PhD on natural language generation with Gerjan @ GroNLP
- 2013-2020: multi-task learning and neural semantic composition at the University of Tübingen
- 2020-2021: freelancing (adapting multi-task learning for companies)
- 2021-2024: machine learning engineer & lead of the performance team @ Explosion AI
- 2024-: machine learning ops engineer (large-scale LLM inference and compute kernels) @ Hugging Face

Gertjan’s contributions to this talk



Gertjan's contributions to this talk

```
subtree(Tree,Tree).
subtree(tree(_,_,Ds,_),Tree) :-
    member(Tree1,Ds),
    subtree(Tree1,Tree).
```



Gertjan's contributions to this talk

```
subtree(Tree,Tree).
subtree(tree(_,_,Ds,_),Tree) :-
    member(Tree1,Ds),
    subtree(Tree1,Tree).
```

```
template<typename T>
T logAdd_(T min, T arg) {
  return std::exp(min - arg);
}

template<typename T, typename... Args>
T logAdd_(T min, T arg, Args... args) {
  return std::exp(min - arg) + logAdd_(min, args...);
}

template<typename T, typename... Args>
T logAdd(T arg, Args... args) {
  auto min = vmin(arg, args...);
  return min - std::log(logAdd_(min, arg, args...));
}
```



Gertjan's contributions to this talk

```
subtree(Tree,Tree).
subtree(tree(_,_,Ds,_),Tree) :-
    member(Tree1,Ds),
    subtree(Tree1,Tree).
```



```
template<typename T>
T logAdd_(T min, T arg) {
  return std::exp(min - arg);
}

template<typename T, typename... Args>
T logAdd_(T min, T arg, Args... args) {
  return std::exp(min - arg) + logAdd_(min, args...);
}

template<typename T, typename... Args>
T logAdd(T arg, Args... args) {
  auto min = vmin(arg, args...);
  return min - std::log(logAdd_(min, arg, args...));
}
```



Gertjan's contributions to this talk

```
subtree(Tree,Tree).
subtree(tree(_,_,Ds,_),Tree) :-
    member(Tree1,Ds),
    subtree(Tree1,Tree).
```

```
if (d->l2_sigma_sq != 0.0) {
  double sigma = 1.0 / d->l2_sigma_sq;
  double f_sq_sum = 0.0;

  for (int i = 0; i < d->dataset->n_features; ++i) {
    if (d->model->f_restrict &&
        !bitvector_get(d->model->f_restrict, i))
      continue;
    f_sq_sum += pow(x[i], 2.0);

    // Impose Gaussian prior on gradient: g_i + x_i / sigma_sq
    g[i] += x[i] * sigma;
  }

  // Impose Gaussian prior on log-likelihood: ll - sum(x) / 2 * sigma_sq
  ll -= f_sq_sum * 0.5 * sigma;
```

```
template
T logAdd
  return
}

template
T logAdd_(T min, T arg, Args... args) {
  return std::exp(min - arg) + logAdd_(min, args...);
}

template<typename T, typename... Args>
T logAdd(T arg, Args... args) {
  auto min = vmin(arg, args...);
  return min - std::log(logAdd_(min, arg, args...));
}
```

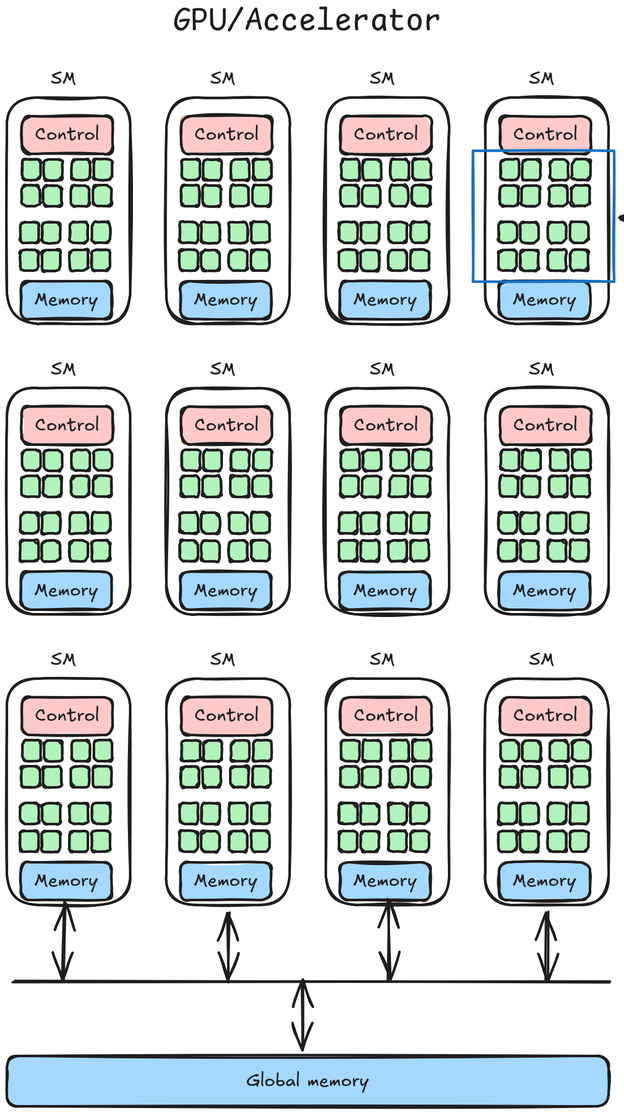




Compute Kernels



What is a compute kernel?



Runs on

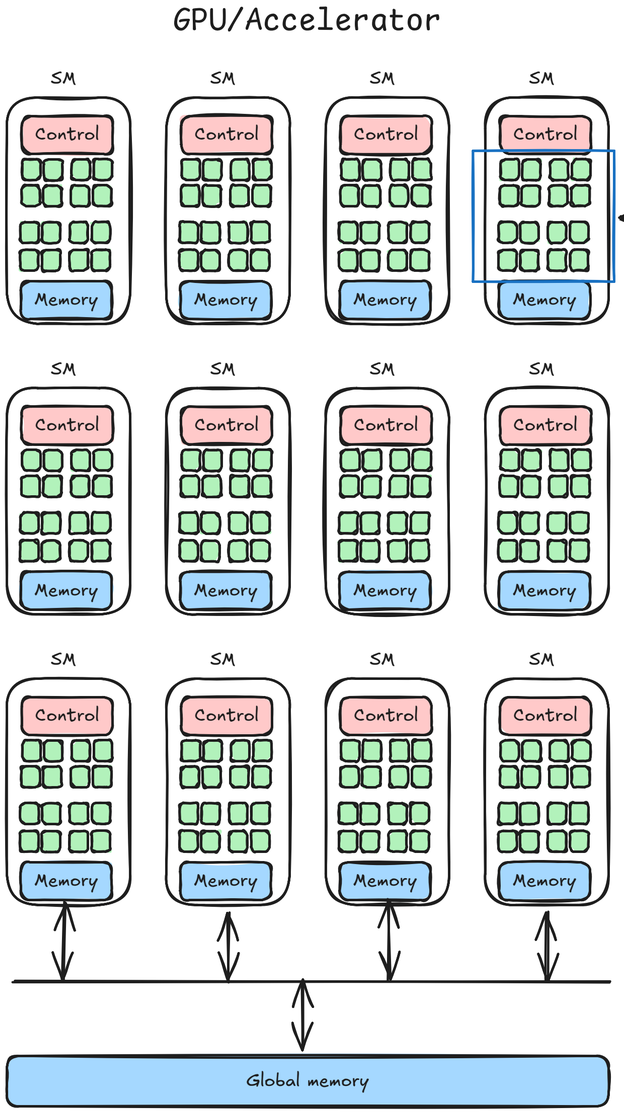
Compute Kernel

- Function
- Runs on GPU
- Many parallel threads

B200:

- 148 Streaming Multiprocessors
- Max 2048 threads per SM
- Max 148 * 2048 = 303,104 threads

What is a compute kernel?



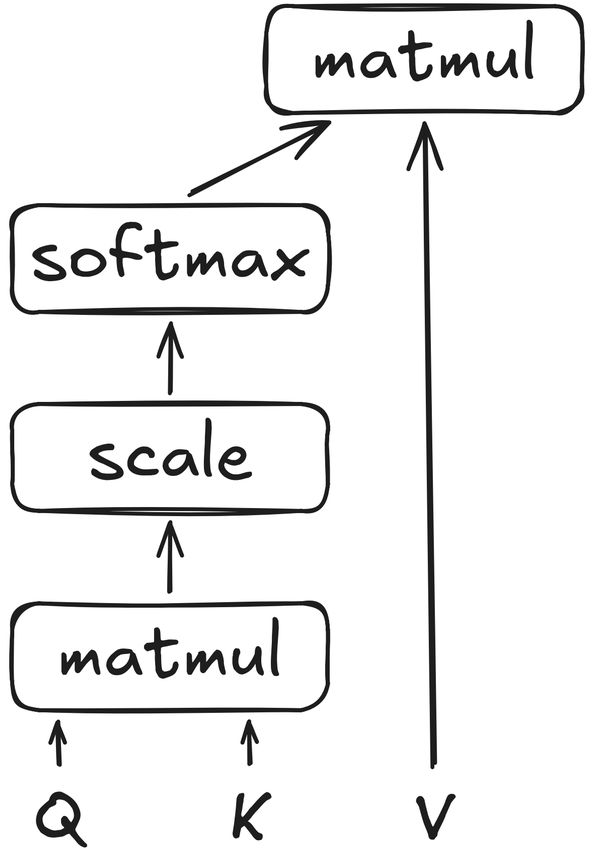
Runs on

Compute Kernel

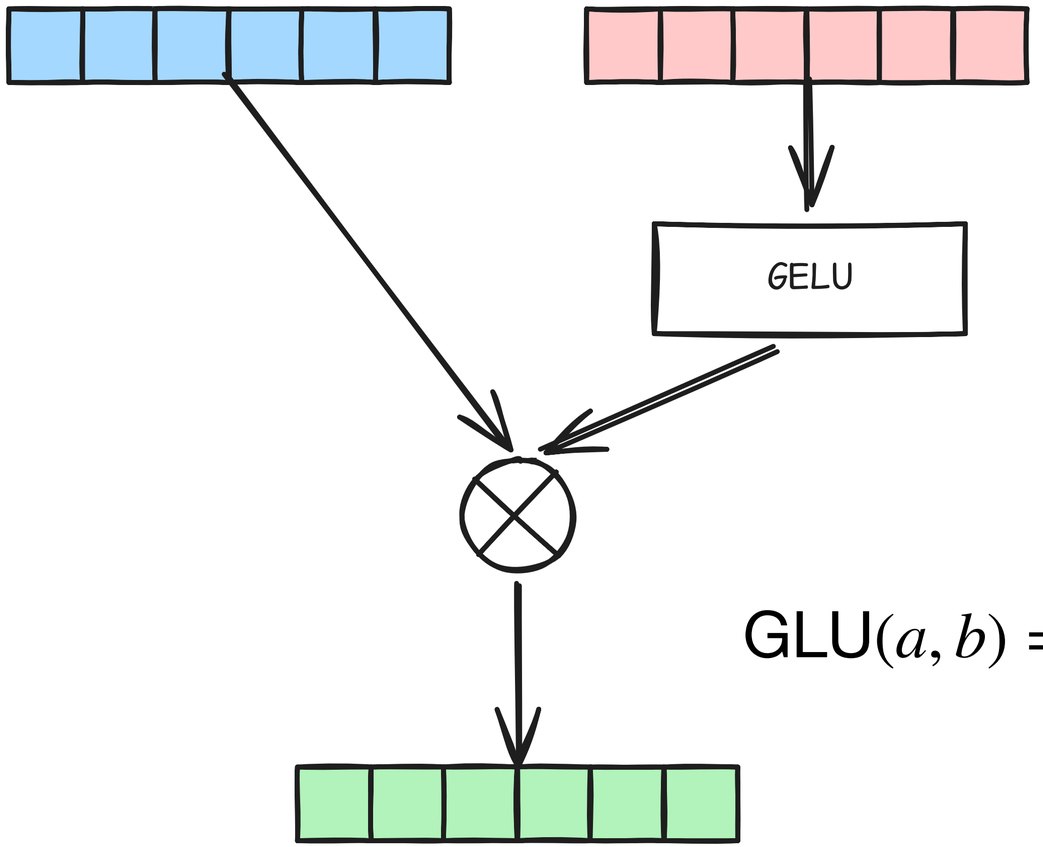
- Function
- Runs on GPU
- Many parallel threads

B200:

- 148 Streaming Multiprocessors
- Max 2048 threads per SM
- Max 148 * 2048 = 303,104 threads



GELU Gated Linear Unit (GEGLU)



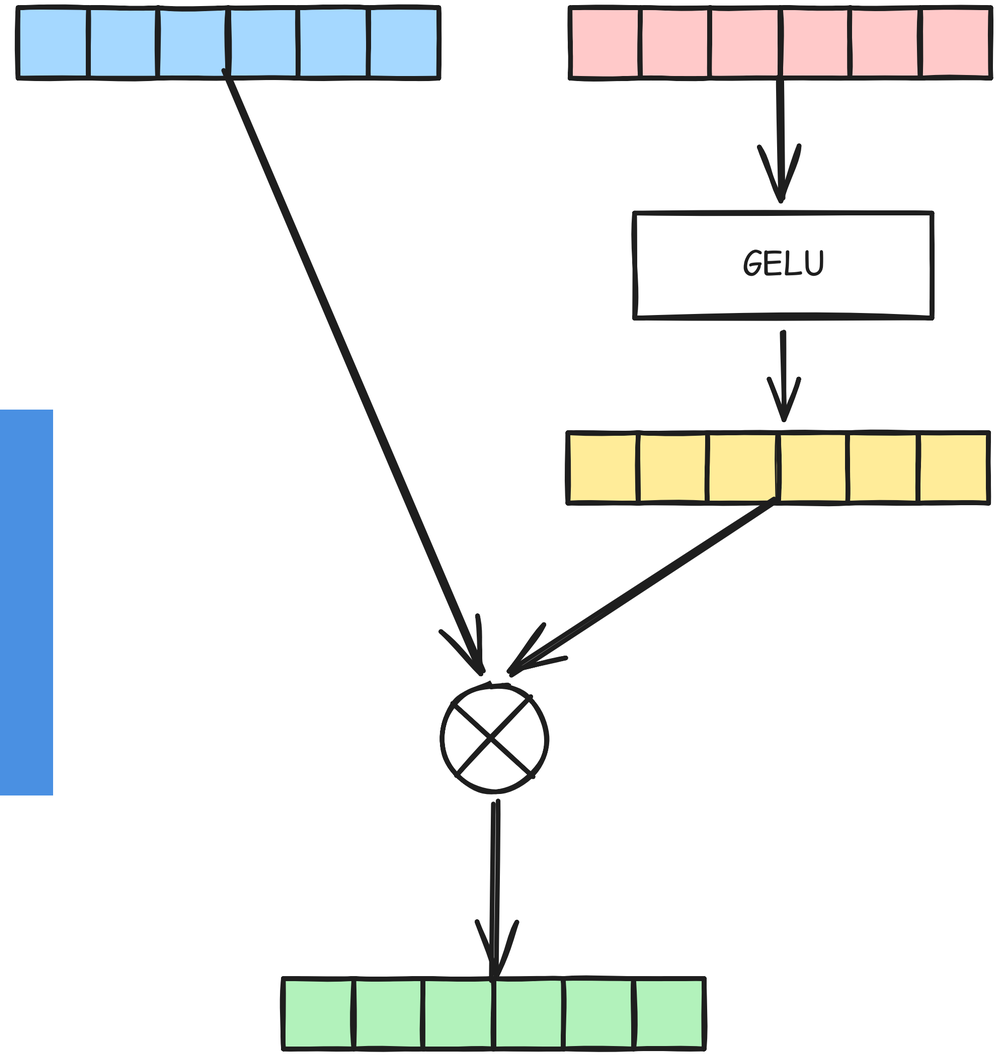
$$\text{GLU}(a, b) = \text{GELU}(a) \otimes b$$

GEGLU in Python

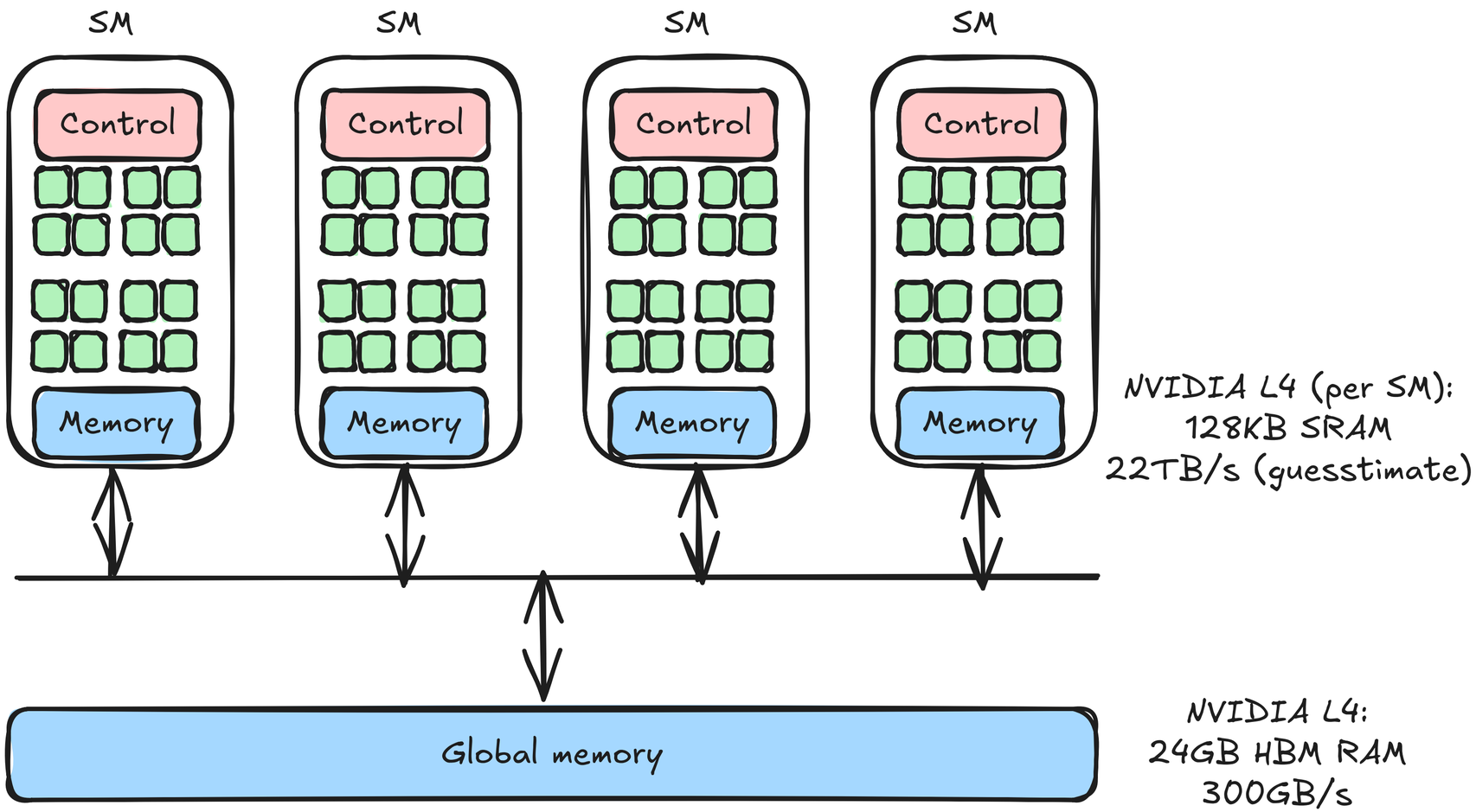
```python
def gelu_and_mul(a: torch.Tensor, b: torch.Tensor) → torch.Tensor:
    return a * F.gelu(b)
```

GEGLU in Python

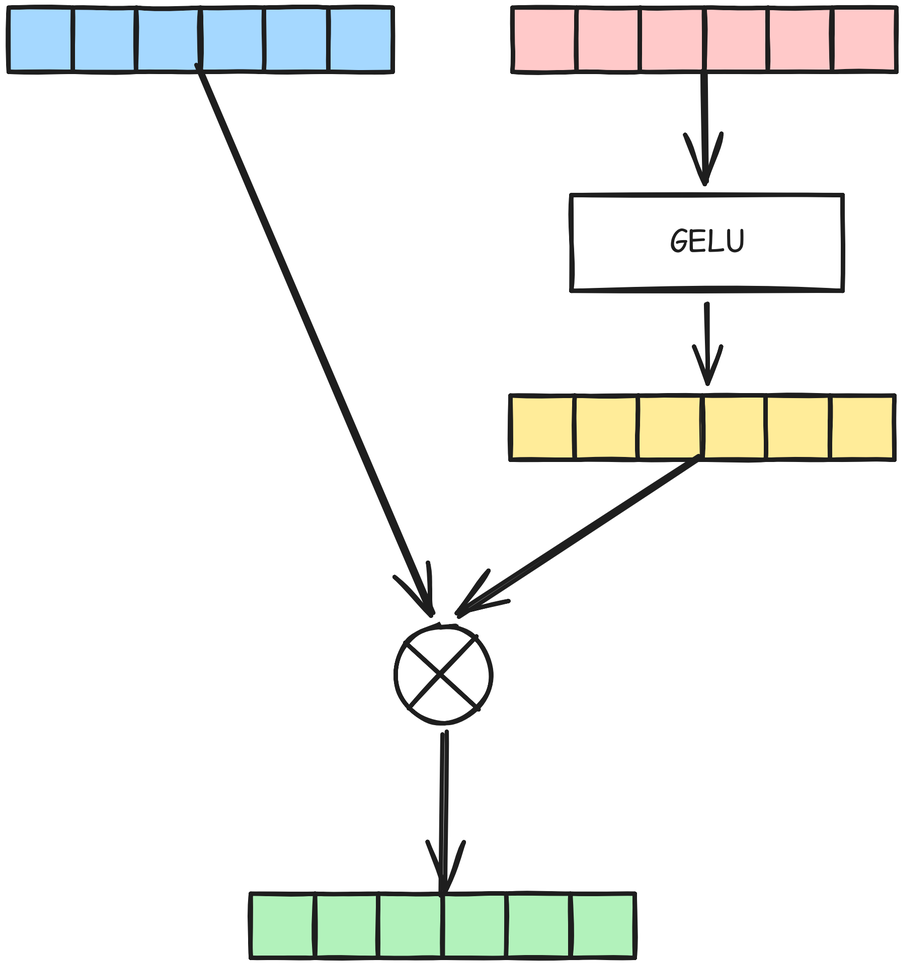
```python
def gelu_and_mul(a: torch.Tensor, b: torch.Tensor) → torch.Tensor:
    return a * F.gelu(b)
```

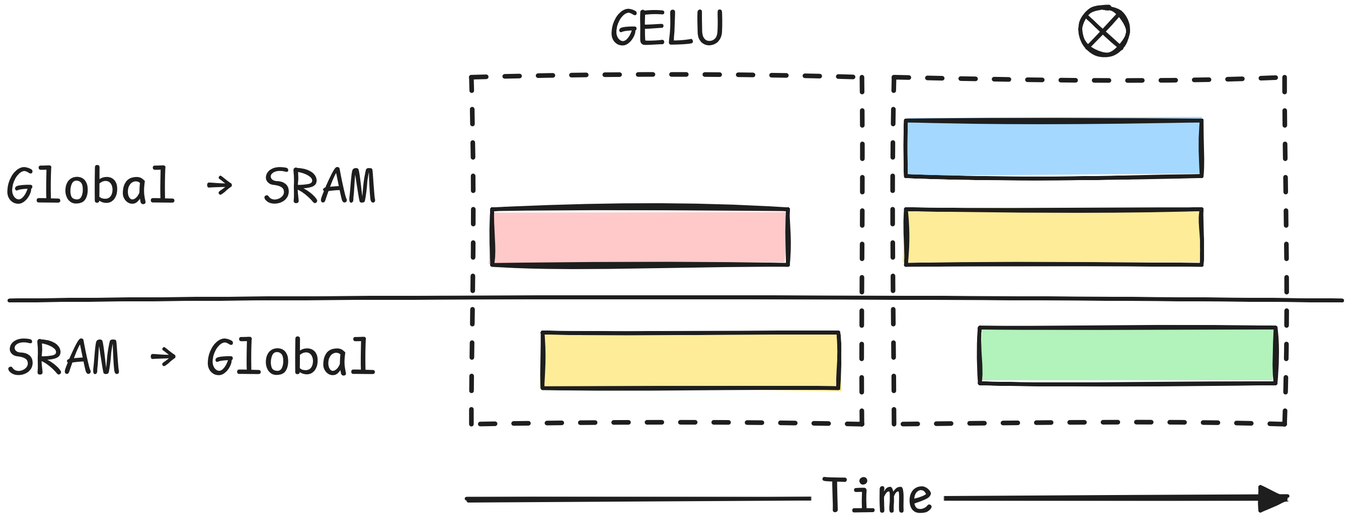


GPU memory hierarchy



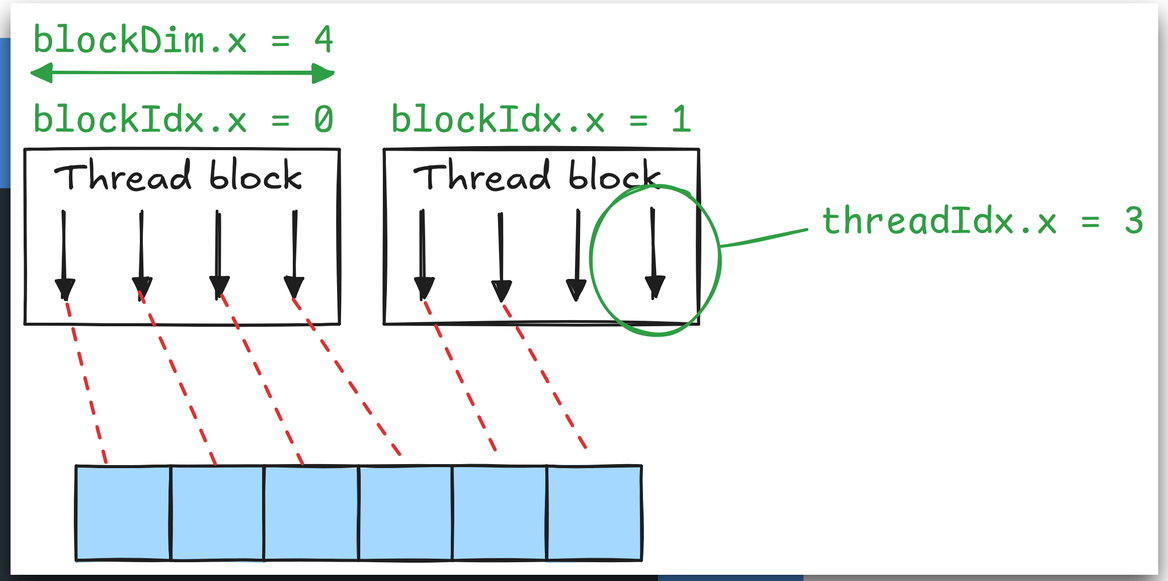
GEGLU Memory Transfers





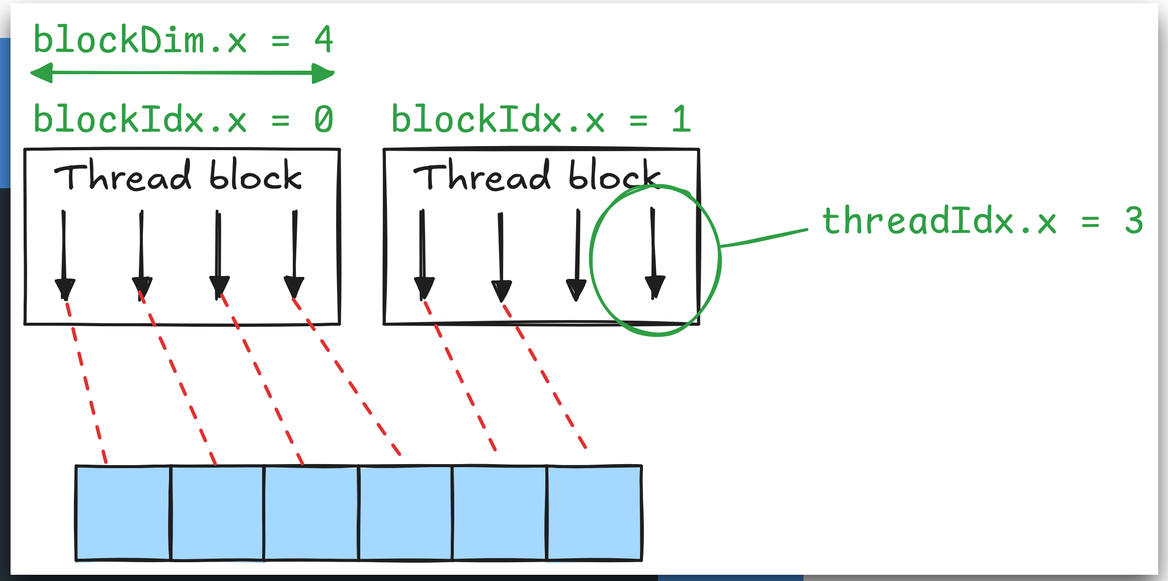
GEGLU CUDA kernel

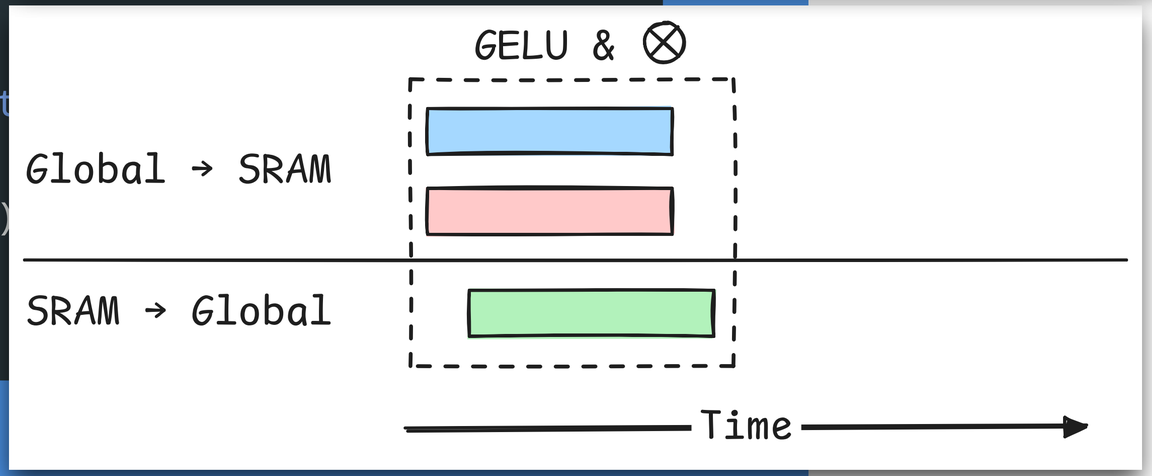
```
template <typename scalar_t>
__global__ void geglu_kernel(
    scalar_t* __restrict__ out,
    const scalar_t* __restrict__ a,
    const scalar_t* __restrict__ b,
    const int64_t n) {
  const int64_t idx = blockIdx.x * (int64_t)blockDim.x + threadIdx.x;
  if (idx < n) {
    out[idx] = a[idx] * gelu_kernel(b[idx]);
  }
}
```



GEGLU CUDA kernel

```
template <typename scalar_t>
__global__ void geglu_kernel(
    scalar_t* __restrict__ out,
    const scalar_t* __restrict__ a,
    const scalar_t* __restrict__ b,
    const int64_t n) {
  const int64_t idx = blockIdx.x * (int64_t
  if (idx < n) {
    out[idx] = a[idx] * gelu_kernel(b[idx])
  }
}
```





GELU

$$\mathrm{GELU}(x) = x \cdot \Phi(x) = x \cdot P(X \leq x),\ X \sim \mathcal{N}(0,1)$$

```
template <typename scalar_t>
__device__ __forceinline__ T gelu_kernel(scalar_t const &x) {
  const float f = (float)x;
  constexpr float ALPHA = M_SQRT1_2;
  return (T)(f * 0.5f * (1.0f + erf(f * ALPHA)));
}
```

GEGLU from Python

```python
def gelu_and_mul(a: torch.Tensor, b: torch.Tensor) -> torch.Tensor:
    out = torch.empty_like(a)
    ops.gelu_and_mul(out, a, b)
    return out
```

Trends in kernels

- Model specialization:
  - Initially very general kernels: matrix multiplication, elementwise operations, scatter/gather.
  - More specialized kernels: FlashAttention, PagedAttention, FlashInfer
  - Model-specific kernels or extensions: attention sinks (GPT-OSS), Multi-head Latent Attention (DeepSeek v2)
- Hardware specialization: specific kernels for GPU architectures (NVIDIA Blackwell, etc.)
- LLM agents are really good at writing and optimizing kernels.
- Combined: quick growth in the number of compute kernels.

Hugging Face Kernels

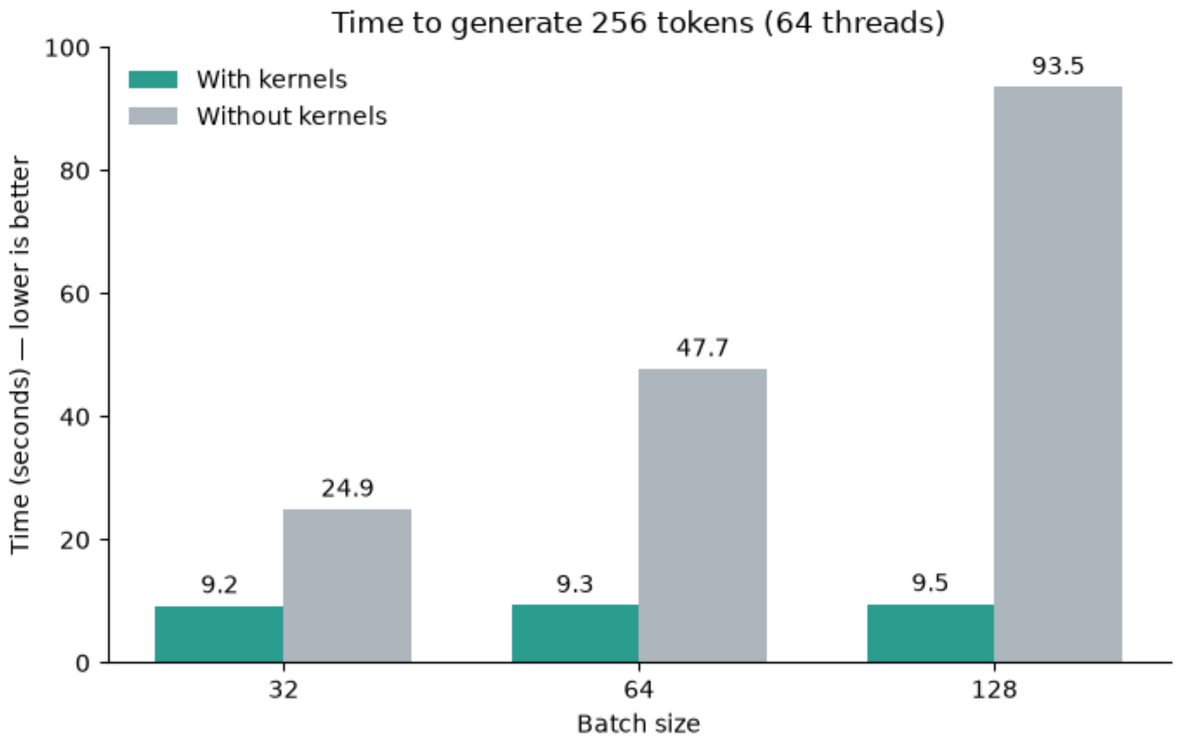


Kernels in Hugging Face

Transformers

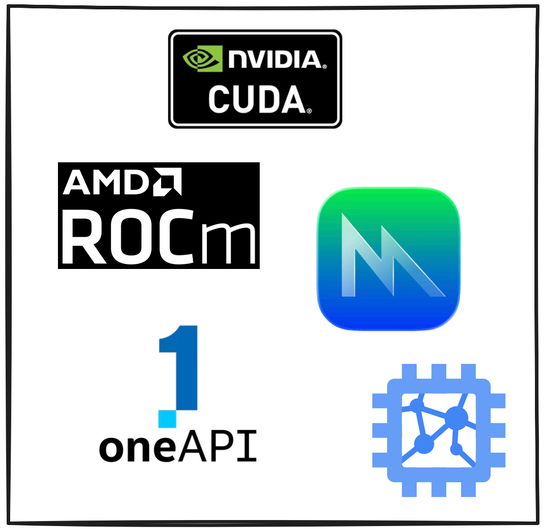
Diffusers

TRL Transformers Reinforcement Learning

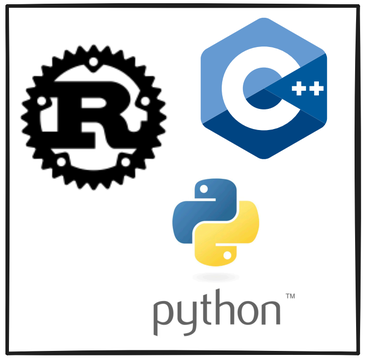


Kernels targets

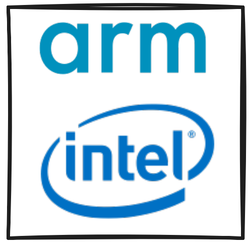
Compute backends



Kernel languages



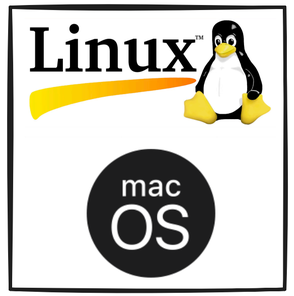
CPU architectures



Frameworks



Operating systems



Additional desiderata

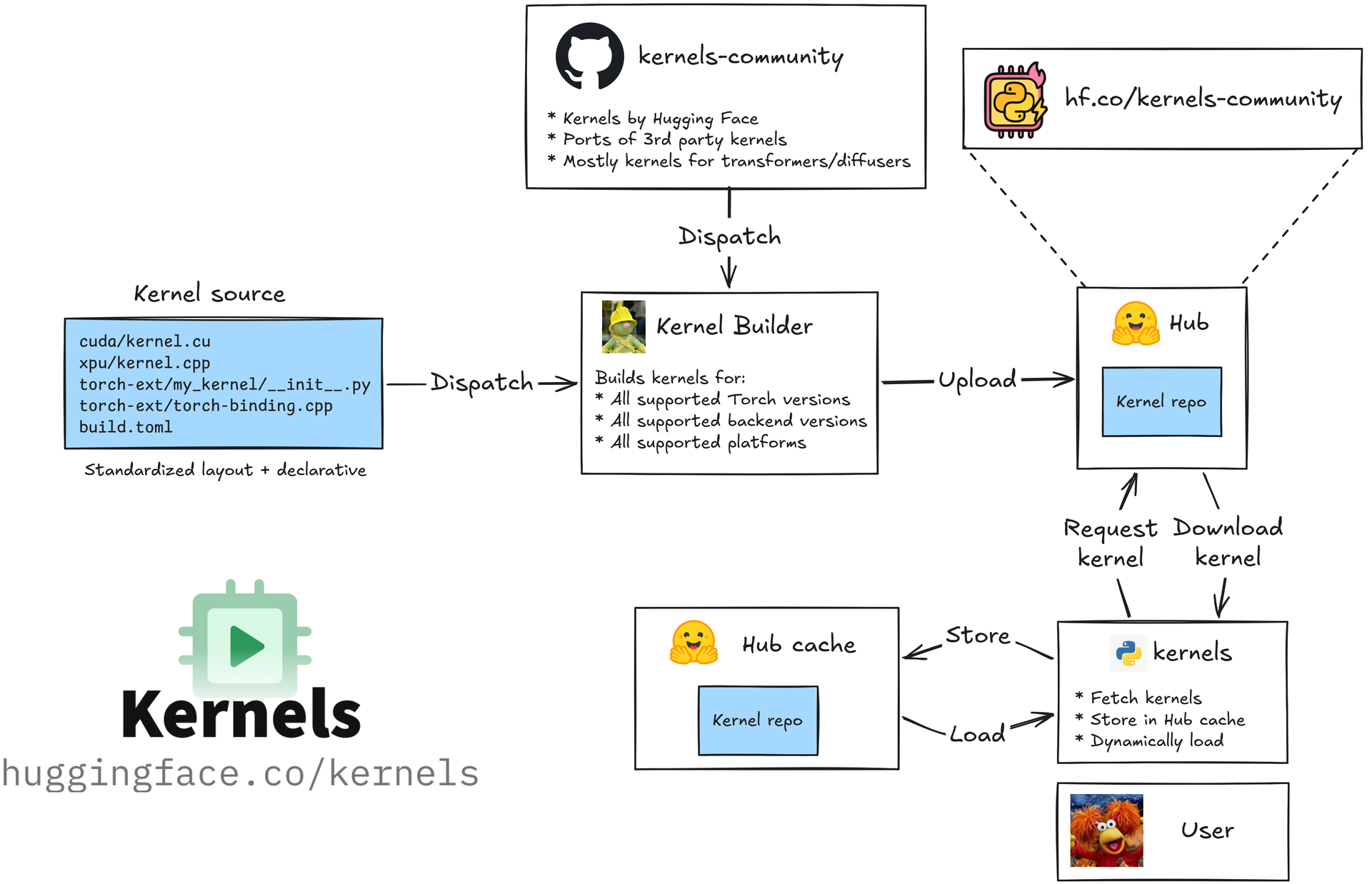
- **Isolation:** multiple versions of the same kernel in a Python process
- **Standalone:** no accidental dependencies
- **Amortized build times:** the user should not have to build.
- **Standardized source:** single builder + mass-maintenance of kernels
- **Security:**
  - **Reproducibility:** make it possible to reproduce the kernel build.
  - **Verification:** Verify integrity through signatures.

What we want

```python
import kernels

# Gets the right build for our: operating system,
# architecture, framework version, compute backend and
# compute backend version
activation = kernels.get_kernel(
    "kernels-community/activation",
    version=1)

# ...
activation.gelu_and_mul(out, x)
```


kernels-community
* Kernels by Hugging Face
* Ports of 3rd party kernels
* Mostly kernels for transformers/diffusers
hf.co/kernels-community
Dispatch
Kernel source
cuda/kernel.cu
xpu/kernel.cpp
torch-ext/my_kernel/__init__.py
torch-ext/torch-binding.cpp
build.toml
Standardized layout + declarative
Dispatch
Kernel Builder
Builds kernels for:
* All supported Torch versions
* All supported backend versions
* All supported platforms
Upload
Hub
Kernel repo
Request kernel
Download kernel
Hub cache
Kernel repo
Store
Load
kernels
* Fetch kernels
* Store in Hub cache
* Dynamically load
User
Kernels
huggingface.co/kernels

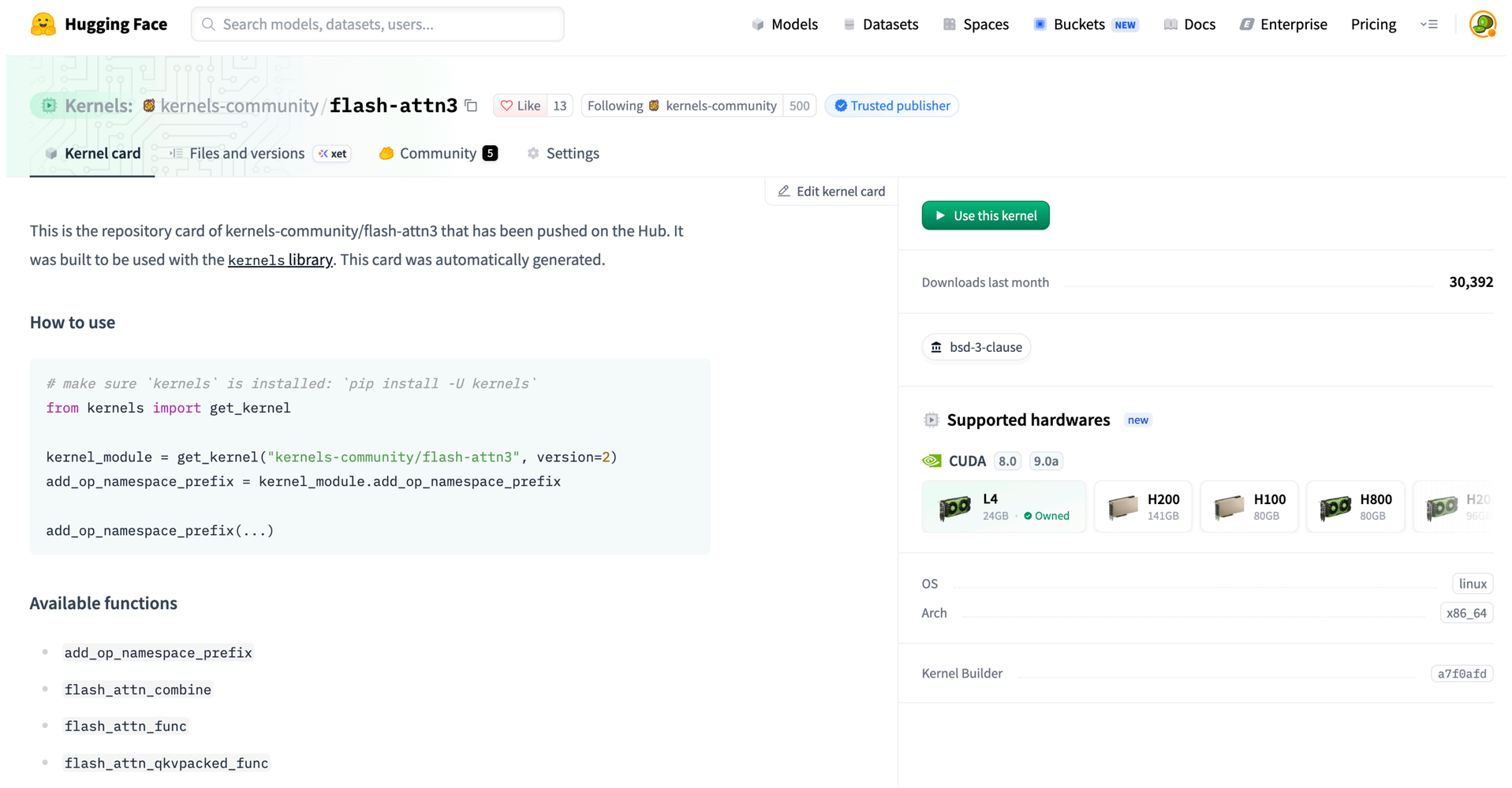
Kernel layers

```python
@use_kernel_forward_from_hub("GeluAndMul")
class GeluAndMul(nn.Module):
    def forward(self, x: torch.Tensor):
        d = x.shape[-1] // 2
        return F.gelu(x[..., :d]) * x[..., d:]

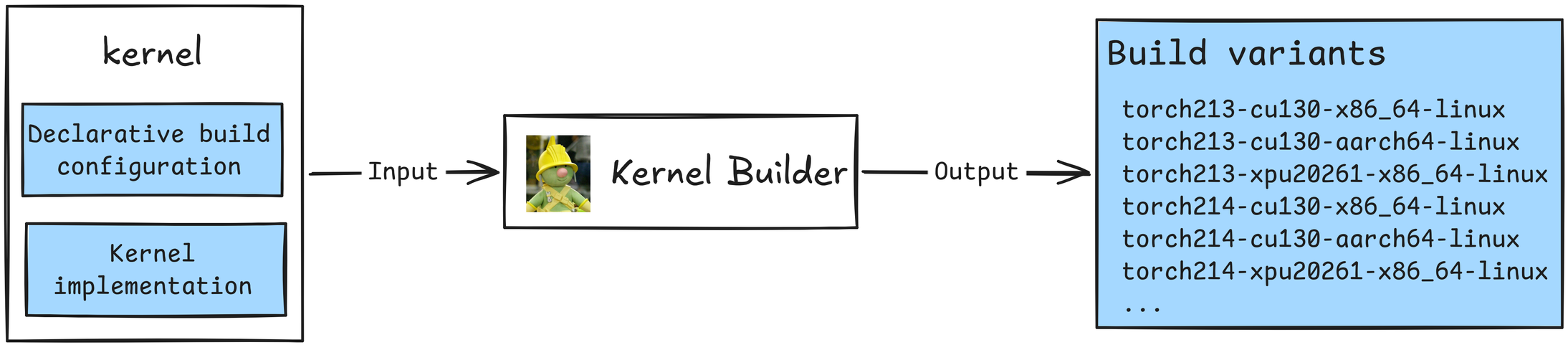
mapping = {
    "GeluAndMul": {
        "cuda": LayerRepository(
            repo_id="kernels-community/activation", layer_name="GeluAndMul", version=1
        )
    }
}

# Model is a model using GeluAndMul
with use_kernel_mapping(mapping):
    kernelize(model, mode=Mode.TRAINING | Mode.TORCH_COMPILE, device="cuda")
```

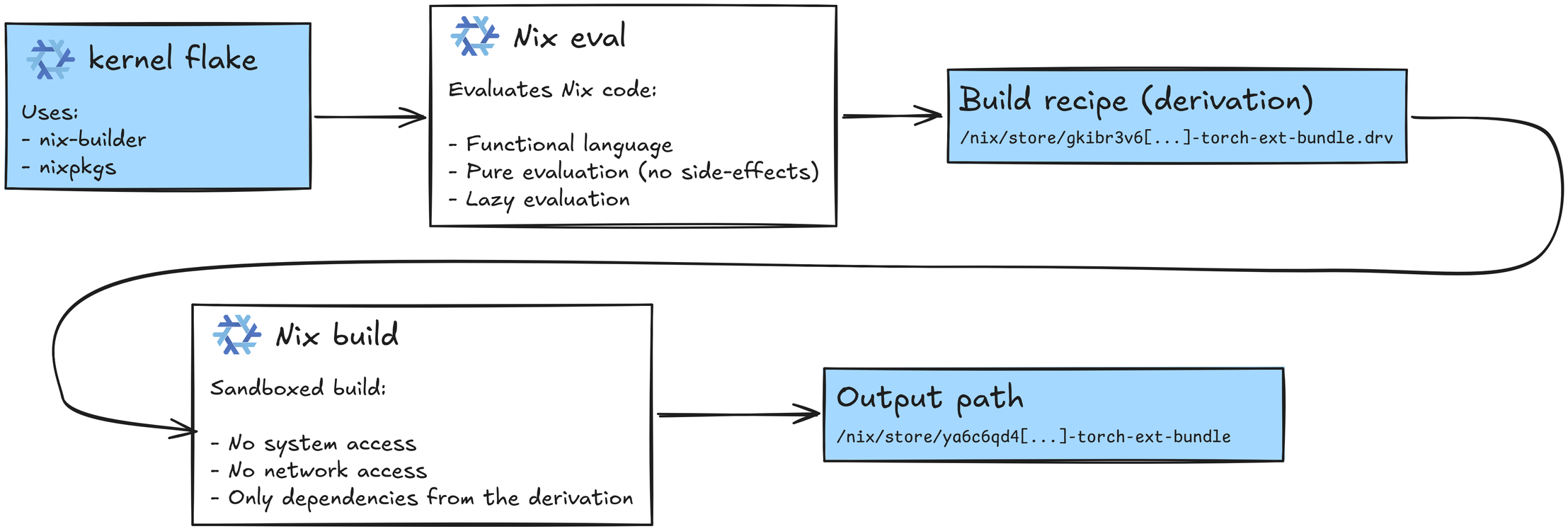
Kernel Hub


Hugging Face
Search models, datasets, users...
Models
Datasets
Spaces
Buckets NEW
Docs
Enterprise
Pricing
Kernels: kernels-community / flash-attn3
Like 13
Following kernels-community 500
Trusted publisher
Kernel card
Files and versions
xet
Community 5
Settings
Edit kernel card
This is the repository card of kernels-community/flash-attn3 that has been pushed on the Hub. It was built to be used with the kernels library. This card was automatically generated.
How to use
make sure `kernels` is installed: `pip install -U kernels`
from kernels import get_kernel
kernel_module = get_kernel("kernels-community/flash-attn3", version=2)
add_op_namespace_prefix = kernel_module.add_op_namespace_prefix
add_op_namespace_prefix(...)
Available functions
add_op_namespace_prefix
flash_attn_combine
flash_attn_func
flash_attn_qkvpacked_func
Use this kernel
Downloads last month
30,392
bsd-3-clause
Supported hardwares
new
CUDA 8.0 9.0a
L4
24GB
Owned
H200
141GB
H100
80GB
H800
80GB
OS
linux
Arch
x86_64
Kernel Builder
a7f0afd

Kernel builder



Hermetic builds



Thank you



Happy Birthday GroNLP!

Extra



Gated Linear Unit (GLU)

$$\text{FFN}(x) = g(xW_1 + b_1)W_2 + b_2$$

$$\text{FFN}(x) = GLU(xW_g + b_g, xW_1 + b_1)W_2 + b_2$$

$$\text{GLU}(a, b) = g(a) \otimes b$$

$$\text{FFN}(x) = GLU(xW_1 + b_1)W_2 + b_2$$

$$\text{GLU}(a) = g(a_{[..n/2)}) \otimes a_{[n/2..)}$$

The roofline model

